Java Notes Part 6

String handling in Java:

- In Java, a **String** is an object that represents a sequence of characters.

Example:

```
String message = "Welcome to Chitkara";
```

- Here, the `message` is a reference variable of type `String`, not a primitive type like `int`, `char`, etc. Instead, it points to a `String` object.

Important points:

- `String` is a **class**, not a primitive data type.
- It belongs to the **java.lang package**, which is automatically imported.
- Strings are created using **double quotes (" ")**.

Example:

```
String name = "Java";
```

Here:

- `name`: reference variable
- `"Java"`: String object stored in memory

Ways to Create String Objects

- There are **two ways** to create String objects in Java.

1 Using String Literals

Example:

```
String s1 = "Hello";

String s2 = "Hello";
```

What happens internally?

- Java stores string literals inside a special memory area called the **String Pool**.
- If a string already exists in the pool, Java **reuses the same object** instead of creating a new one.
- This process is called **String Interning**.

Example:

```
String s1 = "Welcome";
String s2 = "Welcome";

if (s1 == s2)
  System.out.println("same");
else
  System.out.println("not same");
```

Output:

```
same
```

Reason:

- Both variables point to the **same memory location in the String Pool**.

2. Using a new Keyword

Example:

```
String s1 = new String("Welcome");
String s2 = new String("Welcome");

if (s1 == s2)
  System.out.println("same");
else
```

```
    System.out.println("not same");
// Output: not the same
```

- Here, since we use the new operator to create 2 different String class objects, both will be created at different locations inside the heap area, so the output will be “Not Same”.

String Pool

- The **String Pool** is a special area inside heap memory used to store unique string literals.

Benefits:

- Saves memory
- Improves performance
- Reuses existing strings

Example:

```
String a = "Java";
String b = "Java";
String c = "Python";
```

- Both a and b point to the **same "Java" object**.

intern() Method

- The `intern()` method moves a string object to the **String Pool**.

Example:

```
String s1 = "hello";
String s2 = new String("hello");

System.out.println(s1 == s2); // false
```

```
String s3 = s2.intern();
System.out.println(s1 == s3); // true
```

Explanation:

- `s2` was created in the heap
- `intern()` returns reference from **string pool**

Comparing 2 String objects:

- To compare the **content** of two `String` objects, irrespective of how they are created (using string literal or `new` keyword), you should always use the `equals()` method defined in the `String` class. The `equals()` method compares the actual content (character sequence) of the strings, whereas the `==` operator compares the memory references of the objects.

Example:

```
String str1 = "Welcome"; // Created using a string literal
String str2 = new String("Welcome"); // Created using the new keyword

System.out.println(str1 == str2);       // Output: false
// Explanation: '==' compares memory references. Since str1 is in the String Pool and
//str2 is on the heap, the references are different.

System.out.println(str1.equals(str2));  // Output: true
// Explanation: equals() compares the content of the strings.
//Both have the same character sequence, so it returns true.
```

equalsIgnoreCase()

- Ignores case differences.

Example:

```
System.out.println("Java".equalsIgnoreCase("java")); // true
```

Some of the constructors of the String class

- **String()**: Creates an empty String
- **String(byte [] brr)**: Creates a String from the given byte array
- **String(char[] chr)**: Creates a String from a given character array
- **String(String s)**: Creates a String from a given String object
- **String(StringBuilder sb)**: Creates a String from the given StringBuilder object
- **String(StringBuffer sb)**: Creates a String from the given StringBuffer object

Some of the Important String Methods:

1. **char charAt(int position)**: returns the character at the position from the calling string
2. **int length()**: returns the total number of characters in the calling string
3. **byte[] getBytes()**: returns a byte array from the characters of a String
4. **char[] toCharArray()**: returns characters array from the characters of a String

Examples:

```
String s1 = "Java";

System.out.println(s1.charAt(2)); //v
System.out.println(s1.length()); //4

byte brr[] = s1.getBytes();

for(byte b: brr)
  System.out.print(b + " "); //74 97 118 97

System.out.println();

char chr[] = s1.toCharArray();

for(char ch: chr)
  System.out.print(ch + " "); //J a v a
```

5. **static String valueOf(int b)**: returns a String representation of the int value

6. **static String valueOf(float f)**: returns a String representation of the float value.
7. **static String valueOf(boolean b)**: returns a String representation of the boolean value.
8. **static String valueOf(double d)**: returns a String representation of the double value.
9. **static String valueOf(char c)**: returns a String representation of the char value.
10. **static String valueOf(long l)**: returns a String representation of the long value.
11. **static String valueOf(char[] chr)**: returns a String representation of the char[]..
12. **static String valueOf(Object d)**: returns String representation of parameter object value; It calls toString() method

Example1:

```
String statement = "Yogesh of age " + 16 + " years has scored " + 92.25 + "% in class XII";
```

Internally converted as:

```
String statement = "Yogesh of age " + String.valueOf(16) + " years has scored " +
String.valueOf(92.25) + "% in class XII";
```

13. **boolean equals(Object str)**: return true if the parameter and calling object have the same content (matching is case sensitive)
14. **boolean equalsIgnoreCase(String str)**: return true if the parameter and the calling object have the same content (matching is case insensitive)
15. **int compareTo(String str)**: return 0 if the parameter and calling object have the same content (matching is case sensitive); otherwise return the difference of the first character mismatch; otherwise if no character mismatch but any String exhausted then this.length() - str.length()
16. **int compareToIgnoreCase(String str)**: Same as compareTo, but matching is case-insensitive
17. **boolean startsWith(String str)**: returns true if calling String starts with str
18. **boolean endsWith(String str)**: returns true if calling String ends with str

Examples:

```
System.out.println("Java".equals("Java")); //true
System.out.println("Java".equals("java")); //false
System.out.println("Java".equalsIgnoreCase("java")); //true
System.out.println("Java".compareTo("Java")); //0
System.out.println("Java".compareTo("java")); //-32
System.out.println("Java".compareToIgnoreCase("java")); //0
System.out.println("java".compareTo("Java")); //32
System.out.println("javaw".compareTo("java")); //1
System.out.println("java".compareTo("javaw")); //-1
System.out.println("Java".startsWith("Ja")); //true
System.out.println("Java".startsWith("va")); //false
System.out.println("Java".endsWith("va")); //true
System.out.println("Java".endsWith("Ja")); //false
```

19. **int indexOf(char ch)**: returns the first matching index of the string specified by the parameter, or returns -1 if the character is not found.
20. **int indexOf(String str)**: returns the first matching index of the string specified by str, returns -1 if str is not found.
21. **int lastIndexOf(int ch)**: returns the last matching index of the character specified by ch, returns -1 if ch is not found.
22. **int lastIndexOf(String str)**: returns the last matching index of the string specified by str, returns -1 if str is not found.

Examples:

```
System.out.println("bluetooth".indexOf('o')); //5
System.out.println("bluetooth".lastIndexOf('o')); //6
String temp = "value has a value only if the value is valued";
System.out.println(temp.indexOf("value")); //0
System.out.println(temp.lastIndexOf("value")); //39
```

23. **String substring(int startIndex)**: returns the substring starting from startIndex to the end of the string

24. **String substring(int startIndex, int endIndex)**: returns the substring starting from startIndex to the endIndex - 1 of the String
25. **String replace(char original, char replacement)**: returns a new String in which the original character is replaced with a replacement character for the calling string.
26. **String replaceAll(String regex, String replacement):** replace parts of a string that match a regular expression (regex) with another string.
27. **String trim()**: returns a new String with leading and trailing spaces truncated from the calling string.
28. **String toLowerCase()**: returns a new String with all characters in lowercase of the calling string.g
29. **String toUpperCase()**: returns a new String with all characters of the calling string in uppercase.
30. **boolean contains(String s):** checks if the substring exists.

Example:

```
System.out.println("bluetooth".substring(4)); //tooth
System.out.println("bluetooth".substring(0, 4)); //blue
System.out.println("Hook".replace('H', 'L')); //Look
System.out.println(" java ".length()); //9
System.out.println(" java ".trim().length()); //4
System.out.println("Java".toUpperCase()); //JAVA
System.out.println("Java".toLowerCase()); //java
System.out.println("Hello World".contains("World")); //true
System.out.println("Java is fun".replaceAll("fun", "powerfull")); // Java is powerfull
System.out.println("Java".replaceAll("[aeiou]", "*")); //J*v*
System.out.println("123Java456".replaceAll("[0-9]", ""));//Java
```

31. **String format(String s, Object …):** Creates a formatted string.

Example:

```
String name = "John";
```

```
int age = 25;

String msg = String.format("My name is %s and I am %d years old", name, age);
System.out.println(msg);
```

Common format specifiers:

%s: String
%d: Integer
%f: Float
%c: Character
%b: Boolean

32. **static String join(CharSequence delimiter, CharSequence... elements):**

33. **static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements):**

- The `String.join()` method is used to **join multiple strings together using a specified delimiter (separator)**.
- It was introduced in **Java 8**

Example1:

```
String result = String.join("-", "Java", "Python", "C++", "JavaScript");
System.out.println(result);
```

Output:

Java-Python-C++-JavaScript

Example2:

```
String[] languages = { "Java", "Python", "C++" };
String result = String.join(" | ", languages);
```

```
System.out.println(result);
```

Output:

```
Java | Python | C++
```

Tokenizing a String in Java:

- **Tokenizing a string** means **breaking a large string into smaller parts called tokens** based on a delimiter (separator).
- A **delimiter** is a character that separates the tokens, such as:
  - space `" "`
  - comma `","`
  - hyphen `"-"`
  - colon `":"`

  Example string:

  "Java,Python,C++,JavaScript"

  Tokens using `,` delimiter:

  Java

  Python

  C++

  JavaScript

- In Java, tokenizing can be done mainly in two ways:
  1. Using the **StringTokenizer class**
  2. Using the **split() method of the String class**

1. Tokenizing Using `StringTokenizer`

- `StringTokenizer` is a class present in the **`java.util` package**.
- It breaks a string into tokens using a specified delimiter.

```
import java.util.StringTokenizer;
```

Important Methods:

1. **boolean hasMoreTokens():** Checks if more tokens exist
2. **String nextToken():** Returns the next token
3. **int countTokens():** Returns the number of tokens.

Example1: Tokenizing Using Space

- **Default delimiter = space**

```
package com.chitkara;
import java.util.StringTokenizer;
public class Demo {

  public static void main(String[] args) {

      String str = "Java Python C++ JavaScript";

      StringTokenizer st = new StringTokenizer(str);

      while(st.hasMoreTokens()) {
          System.out.println(st.nextToken());
      }
  }
}
```

Output:

```
Java

Python

C++

JavaScript
```

Example 2: Tokenizing Using Comma

```java
package com.chitkara;
import java.util.StringTokenizer;
public class Demo {

	public static void main(String[] args) {

		String str = "Java,Python,C++,JavaScript";

		StringTokenizer st = new StringTokenizer(str, ",");

		System.out.println("Total Tokens: " + st.countTokens());

		while (st.hasMoreTokens()) {
			System.out.println(st.nextToken());
		}
	}
}
```

2. Tokenizing Using the `split()` Method

- The `split()` method belongs to the `String` class.
- It splits a string based on a delimiter and returns an array of strings.

Syntax:

```
String[] arr = string.split(delimiter);
```

Example 1: Tokenizing Using Comma

```
package com.chitkara;
public class Demo {

	public static void main(String[] args) {

		String str = "Java,Python,C++,JavaScript";

		String[] tokens = str.split(",");

		for (String s : tokens) {
			System.out.println(s);
		}
	}
}
```

Example 2: Tokenizing Using Space

```
package com.chitkara;
public class Demo {

	public static void main(String[] args) {

		String str = "Java Python C++ JavaScript";

		String[] words = str.split(" ");

		for (String word: words) {
			System.out.println(word);
		}
	}
}
```

When to Use Which?

Use **StringTokenizer** when:

- Simple tokenization is needed
- Working with legacy Java code

Use **split()** when:

- Regex support is required
- Working with modern Java programs

Immutability Concept of String object:

- Immutability refers to the property of an object whose state cannot be changed after it is created. The String class in Java is **immutable**, which means that once a String object is created, it cannot be modified. If you perform any operation on a String, a new String object is created rather than modifying the original one.

Example:

```
String message = "Welcome";
message.concat(" user");
System.out.println(message);
```

Output:

Welcome // Original String is unchanged.

Reason: Here, we are using the **concat()** method to add another string to the previous string. Since string objects are immutable, if we call any method to make any modification in the string object, that method will return a new string object with modified content.

Example:

```
String message = "Welcome";
```

```
String newMessage = message.concat(" user");
System.out.println(message);
System.out.println(newMessage);

//Another Example
String s1 = "Welcome";
s1 = s1.concat(" user");
System.out.println(s1);
//The immutability applies to the String object stored in memory, not the
//reference variable of the String type.
```

Why is String Immutable in Java?

- In Java, the **`String` class is immutable**, which means that once a `String` object is created, **its value cannot be changed**. If any modification is performed on a string, a **new String object is created instead of modifying the existing one**.
- The main reasons for making `String` immutable in Java are as follows:

1. Security

- Strings are widely used in **security-sensitive operations**, such as storing **file paths, database URLs, network connections, usernames, and passwords**. If strings were mutable, their values could be changed by malicious code after validation, which could lead to security vulnerabilities. Immutability ensures that once a string is created, its value cannot be altered, making the system more secure.

2. String Pooling (Memory Efficiency)

- Java uses a special memory area called the **String Constant Pool** to store string literals. When multiple variables refer to the same string value, Java stores only **one copy of that string in memory**. Since strings are immutable, different parts of the program can safely share the same object without worrying about unexpected modifications. This **improves memory efficiency and performance**.

3. Thread Safety

- Because `String` objects cannot be modified after creation, they are **inherently thread-safe**. Multiple threads can access the same string object simultaneously without causing data inconsistency or requiring synchronization. This simplifies concurrent programming and improves performance in multi-threaded environments.

In summary, making `strings` immutable improves **security, memory efficiency through string pooling, and thread safety**, which are essential for reliable and efficient Java applications.

StringBuffer and StringBuilder Classes in Java:

- In Java, the **String class is immutable**, meaning that once a string object is created, its value **cannot be changed**. If we try to modify a string, a **new object is created in memory**.
- Because of this behavior, frequent string modifications can create **many unnecessary objects**, which may reduce performance.
- To solve this problem, Java provides two **mutable companion classes**:
  - `StringBuffer`
  - `StringBuilder`
- Both classes belong to the **`java.lang` package** and allow **modifying the same object instead of creating new ones**, which improves performance when performing many string operations.

1. StringBuilder Class

- The **StringBuilder class** is used to **create and manipulate mutable strings**.
  It provides methods for appending, inserting, replacing, deleting, and reversing **characters** in a string.
- `StringBuilder` is **not thread-safe**, which makes it **faster** than `StringBuffer`.

Example:

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb);
```

Output:

```
Hello World
```

2. StringBuffer Class

- `StringBuffer` works almost the same as `StringBuilder`. The main difference is that **StringBuffer is thread-safe**, meaning it is safe to use in **multi-threaded environments**.
- However, because of synchronization, **StringBuffer is slower than StringBuilder**.

Example

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb);
```

Output:

```
Hello World
```

Difference Between StringBuilder and StringBuffer:

Feature	StringBuilder	StringBuffer
Thread Safety	Not thread-safe	Thread-safe
Performance	Faster	Slower
Use Case	Single-threaded programs	Multi-threaded programs

When to Use

Use StringBuilder when:

- The program runs in a **single-threaded environment**
- **Performance is important**

- Thread safety is not required.

Use StringBuffer when:

- The program runs in a **multi-threaded environment**
- **Thread safety is required.**

Creating StringBuilder Objects

1. Creating an Empty StringBuilder

```
StringBuilder builder = new StringBuilder();
```

Default Capacity: 16 characters

2. Creating a StringBuilder with Custom Capacity

```
StringBuilder builder = new StringBuilder(50);
```

- This creates a `StringBuilder` object with a **capacity of 50 characters**.

Common Methods of StringBuffer / StringBuilder

- Both classes provide many useful methods for modifying strings.

1. append()

- Adds text at the **end of the string**.

```
StringBuffer sb = new StringBuffer("Java");

sb.append(" Programming");
System.out.println(sb);
```

Output:

Java Programming

2. insert()

- Inserts text at a **specific position**.

```
StringBuffer sb = new StringBuffer("Java");
sb.insert(4, " Language");
System.out.println(sb);
```

Output

Java Language

3. replace()

- Replaces characters **between two indexes**.

```
StringBuffer sb = new StringBuffer("Hello World");
sb.replace(6, 11, "Java");
System.out.println(sb);
```

Output

Hello Java

4. delete()

- Deletes characters between indexes.

```
StringBuffer sb = new StringBuffer("Java Programming");
sb.delete(4, 16);
System.out.println(sb);
```

Output

```
Java
```

5. reverse()

- Reverses the characters of the string.

```
StringBuffer sb = new StringBuffer("Java");
sb.reverse();
System.out.println(sb);
```

Output

```
avaJ
```

6. capacity()

- Returns the **total capacity of the buffer**.

```
StringBuffer sb = new StringBuffer();
System.out.println(sb.capacity());
```

Output

16

7. length()

- Returns the **number of characters currently stored** in the object.

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.length());
```

Output

4

8. ensureCapacity()

- Increases the capacity if required.

```
StringBuffer sb = new StringBuffer();
sb.ensureCapacity(50);
```

- This ensures the buffer can hold **at least 50 characters**.

9. charAt()

- Returns the character at a **specific index**.

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.charAt(2));
```

Output

v

10. setCharAt()

- Changes the character at a **specific index**.

```
StringBuffer sb = new StringBuffer("Java");
sb.setCharAt(0, 'K');
System.out.println(sb);
```

Output

```
Kava
```

Capacity vs Length in StringBuffer / StringBuilder

- When working with StringBuffer or StringBuilder, two important concepts are:
    - **Length:** Length represents the number of characters currently stored in the object.
    - **Capacity:** Capacity represents the maximum number of characters that can be stored in the object before the buffer needs to resize itself.

- StringBuffer and StringBuilder allocate extra memory so that frequent resizing can be avoided.

Example:

```
StringBuffer sb = new StringBuffer("Java");
System.out.println("Length: " + sb.length());
System.out.println("Capacity: " + sb.capacity());
```

Output

Length: 4

Capacity: 20

Explanation

- The string "Java" contains 4 characters, therefore:

  Length = 4

- When a `StringBuffer` or `StringBuilder` object is created using a **String**, Java calculates the capacity using the following formula:

  Default Capacity Formula

  Capacity = 16 + length of the string

Capacity Expansion in StringBuffer / StringBuilder

- Both **StringBuffer** and **StringBuilder** maintain an **internal buffer capacity**.
- If the number of characters stored in the object **exceeds the current capacity**, Java automatically **increases the capacity** of the buffer.
- This process is called **Capacity Expansion**.

Capacity Expansion Formula

- When the current capacity is exceeded, Java increases the capacity using the following formula:

  New Capacity = (Old Capacity × 2) + 2

Example

- Suppose the current capacity is:

  Old Capacity = 20

- If more characters are added and the buffer becomes full, Java increases the capacity as follows:

  New Capacity = (20 × 2) + 2

  = 40 + 2

  = 42

- So the new capacity becomes:

  42 characters

Test.java:

```
public class Test {

  public static void main(String[] args) {

    StringBuffer sb = new StringBuffer("Java");

    System.out.println("Initial Length: " + sb.length());
    System.out.println("Initial Capacity: " + sb.capacity());

    sb.append(" Programming Language");

    System.out.println("After Append Length: " + sb.length());
    System.out.println("After Append Capacity: " + sb.capacity());
  }
}
```

Output:

Initial Length: 4

Initial Capacity: 20

After Append Length: 24

After Append Capacity: 42

Explanation:

Initial capacity = 16 + 4 = 20

After adding more characters: capacity exceeded

New capacity = (20 × 2) + 2 = 42

Converting String to Primitive Types:

```
String num = "123";
int n = Integer.parseInt(num);
double d = Double.parseDouble("3.14");
long l = Long.parseLong("12345");
boolean b = Boolean.parseBoolean("true");
```

Converting Primitive to String:

- **Using `valueOf()`:** Recommended to use

```
int age = 25;
String s = String.valueOf(age);
```

- **Using + operator:**

```
int age = 10;
String s = age+"";

System.out.println(s);
```

Java internally does something similar to:

```
new StringBuilder().append(age).append("").toString();
```

Text-Block in Java: (New feature introduced in Java 15)

- Java 15 introduced a new type of string literal called a **text-block** which allows programmers to preserve indents and multiple lines without the need to add white spaces within quotes.
- In Java, we can create a **text block** by enclosing the text in the **triple quote("""").**

Example:

```
String html = """
                <html>
                <body>
                <h1>Hello</h1>
                </body>
                </html>
                """;
}

        System.out.println(htmlCode);
```

- You can also use **text blocks** to create formatted Strings, using placeholders like **%s** or **%d.**

Example:

```
        String name = "Rahul";
        int age = 30;

        String message = """
                        Hello, my name is %s
                        And I am %d years old.
                """;
        String formatedString = String.format(message, name, age);
        System.out.println(formatedString);
```

Student Task:

1. **Predict the Output:**

```
String s1 = "Java";

String s2 = "Java";
System.out.println(s1 == s2);
```

A. true

B. false

C. Compilation error

D. Runtime error

2. **Predict the Output:**

```
String s1 = new String("Java");

String s2 = new String("Java");
System.out.println(s1 == s2);
```

A. true

B. false

C. Compilation error

D. Runtime error

3. **Predict the Output:**

```
StringBuffer sb1 = new StringBuffer("Java");

StringBuffer sb2 = new StringBuffer("Java");
System.out.println(sb1.equals(sb2));
```

A. true

B. false

C. Compilation error

D. Runtime error

4. Predict the Output:

```
String s1 = "Java";

String s2 = new String("Java");
System.out.println(s1.equals(s2));
```

A. true

B. false

C. Compilation error

D. Runtime error

5. Predict the Output:

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.length());
System.out.println(sb.capacity());
```

A. 4 20
B. 4 16
C. 4 4
D. 20 4

6. **Predict the Output:**

```
String s = "Java";
StringBuilder sb = new StringBuilder(s);
sb.reverse();
System.out.println(s);
```

A. avaJ

B. Java

C. Compilation error

D. Runtime error

7. **Predict the Output:**

```
StringBuilder sb = new StringBuilder("Java");
sb.delete(1,3);
System.out.println(sb);
```

A. Ja

B. Jva

C. J

D. Java

Explanation:

`delete(start, end)` removes characters from **start index to end-1**.

8. **Which statement is correct?**

A. String is mutable
B. StringBuilder is thread-safe
C. StringBuffer is thread-safe
D. StringBuilder is immutable

9. Check if a String is a Palindrome

- Write a Java program to **check whether a string is a palindrome using** `StringBuilder`.

Solution:

```
package com.chitkara;
public class Demo {
	public static void main(String[] args) {

		String str = "madam";
		StringBuilder sb = new StringBuilder(str);
		String reversed = sb.reverse().toString();

		if (str.equals(reversed))
			System.out.println("Palindrome");
		else
			System.out.println("Not Palindrome");
	}
}
```

10. Remove Vowels from a String using StringBuffer:

Solution:

```
package com.chitkara;
public class Demo {
	public static void main(String[] args) {

		StringBuffer sb = new StringBuffer("Java Programming");
	for(int i = 0; i < sb.length(); i++) {
	  char ch = sb.charAt(i);
```

```
            if("aeiouAEIOU".indexOf(ch) != -1) {
                sb.deleteCharAt(i);
                i--;
            }
        }
        System.out.println(sb);
            }
    }
```

11. **Replace Spaces with Hyphens using StringBuilder:**
    - Write a Java program to **replace all spaces in a string with '-' using StringBuilder**.

Solution:

```
package com.chitkara;

public class Demo {

        public static void main(String[] args) {

            String str = "Java Programming Language";
            StringBuilder sb = new StringBuilder(str);

            for(int i = 0; i < sb.length(); i++) {
                if(sb.charAt(i) == ' ') {
                    sb.setCharAt(i, '-');
                }
            }
            System.out.println(sb);
        }

}
```